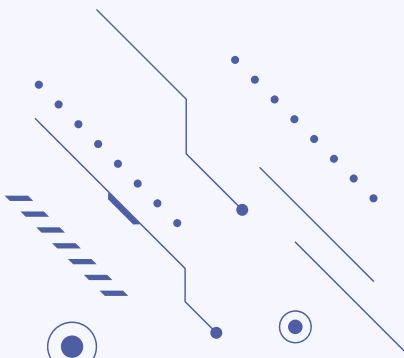


CI/CD Intégration & Déploiement Continu

BESOMBE Tristan



AVANT TOUT

Qui suis-je ?

- Ancien élève EPSI
- Consultant DevOps chez Graphicient
- Ni votre pote, ni votre père



Partage

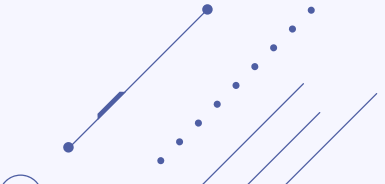


Agréable

AVANT TOUT



Et vous ?

- Alternance ? Quelle entreprise ?
 - Pourquoi vous êtes la ?
 - Vos passions/hobbies ?
- 
- 

AVANT TOUT



Temps

Arrivée à l'heure
2 minutes max
Porte fermé = attendre
la pause



Rendu

Respecter les
deadlines



Autonomie

Utilisez l'IA
en tant qu'outil,
vous ne l'aurez pas
toujours...

SOMMAIRE

01

Introduction

02

CI/CD

03

Env & sécurité

04

TP

The background features decorative geometric patterns in the corners, consisting of thin blue lines, dots, and circles. In the top-left, there are several parallel lines and a small cluster of dots. In the top-right, a circle with a dot inside is visible. In the bottom-left, there are more parallel lines and a circle with a dot. In the bottom-right, there are several parallel lines and a circle with a dot. A horizontal line with dots at its ends is positioned below the '01' text.

01

Avant le CI/CD

On faisait comment avant ?

AVANT LE CI/CD

Comment on déployait avant



Scripts bash fragiles
"ça marche sur ma machine"



SSH direct en prod
Modifs sans
traçabilité, sans
retour arrière



FTP / SCP manuel
Upload direct sur le
serveur de prod



Procédures Word
Runbook de 40
pages, suivi à la main

POURQUOI C'ÉTAIT RISQUÉ

Deux catégories de problèmes

Humain

Erreur de manipulation

Oubli d'une étape

Dépendance

Pas de validation

Technique

Pas de versioning

Pas de tests auto

Logs inexistants

Secrets en clair

CASSER LES SILOS

Avant : deux mondes séparés

- Dev : "ça marche chez moi, déployez mieux"
- Ops : "vous n'avez qu'à mieux coder"
- Résultat : incidents fréquents, blame game, pas de responsabilité partagée
- Après la CI/CD :
 - Responsabilité commune du code jusqu'à la production
 - Objectif partagé : livrer de la valeur rapidement et en sécurité



02

CI/CD

*Est-ce que le code est propre, testable et
intégrable ?*

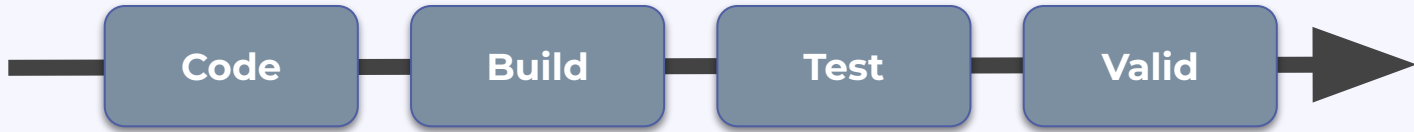
PIPELINE CI/CD

Déploiement de A -> Z



PIPELINE CI

La CI vérifie automatiquement que le code peut être intégré sans casser le projet.



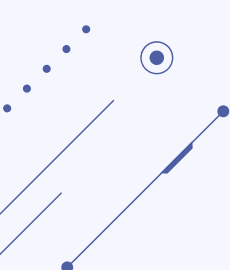
Exemples d'outils :

- GitHub Actions
- GitLab CI
- Jenkins
- Azure DevOps

PIPELINE CD

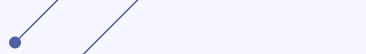
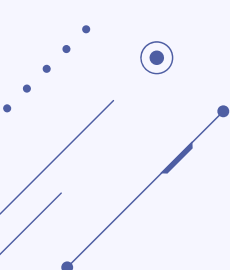
La CD récupère une version validée et
l'envoie vers un environnement.





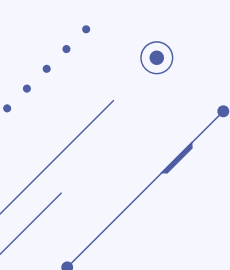
WORKFLOW CI/CD

- **Push / PR** : commit déclenche le pipeline automatiquement
 - **Build** : compilation, image Docker, packaging du livrable
 - **Test** : unitaires, intégration, lint, qualité de code
 - **Deploy staging** : déploiement automatique sur l'env de test
 - **Approbation** : validation manuelle (Continuous Delivery)
 - **Deploy prod** : mise en production + smoke tests + monitoring
- 



CAS CONCRET

1. Un dev pousse son code
 2. GitHub Actions se lance
 3. L'application est buildée
 4. Les tests passent
 5. Une image Docker est créée
 6. L'image est envoyée dans un registry
 7. L'application est déployée en staging à partir de l'image
 8. Si tout est ok sur staging, validation manuelle
 9. L'application est déployée en production
- 



VOCABULAIRE

Pipeline

Enchaînement
d'étapes
automatisées

Step

Tâche dans un job

Action

Action ou plugin
réutilisable



Job

Unité d'exécution

Trigger

Événement
déclencheur

Reproductibilité

Toujours même
résultat

GITHUB ACTIONS

- on = quand le pipeline se lance
- jobs = ce qu'on veut faire
- steps = les étapes exécutées
- run = commande shell

Gratuit jusqu'à 2 000 min/mois, le fichier est versionné avec votre code.

```
on:
  push:
    branches: [main]
  pull_request:

jobs:
  build-test:
    runs-on: ubuntu-latest

    steps:
      - name: Build image Docker
        run: docker build -t nginx-ci-demo .

      - name: Tester la configuration Nginx
        run: docker run --rm nginx-ci-demo nginx -t

      - name: Lancer le conteneur
        run: |
          docker run -d --name nginx-test -p
          8080:80 nginx-ci-demo
```

JENKINS

Plus de flexibilité, plus de responsabilité

Cas d'usage : entreprises avec infra on-premise.

- Serveur auto-hébergé : vous gérez la machine
- Jenkinsfile en Groovy : Declarative ou Scripted
- +1800 plugins : intégration avec tout l'écosystème
- Agents : nœuds dédiés par type de build (Java, Docker...)

The slide features decorative geometric patterns in the corners, consisting of thin blue lines, dots, and circles. A horizontal line with dots at its ends is positioned above the main title.

03

Env & Sécurité

*Ce que vous mettez dans votre pipeline, c'est
de la production.*

ENVIRONNEMENTS

On ne déploie pas directement en production sans étape de validation.

Dev

Environnement
local ou partagé

Test

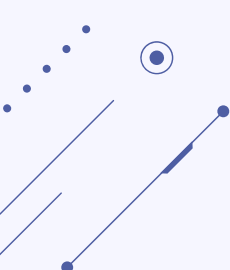
Validation
technique

Staging

Environnement
proche de la
production

Production

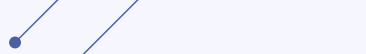
Application
officielle



SECURITE



Ce que l'on voit trop souvent :

- `DB_PASSWORD=admin123` dans le code
 - `API_KEY="sk-..."` committée sur GitHub
 - Credentials AWS dans le Dockerfile
 - Token Slack en clair dans le script
- 

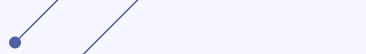
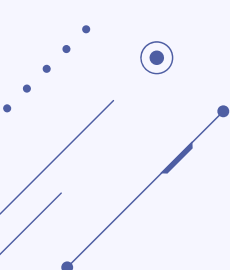
*Un secret commité = fuite de données
garantie.*



GÉRER LES SECRETS

Règle d'or : *un secret ne doit JAMAIS apparaître dans un log, un artifact ou un repo.*

Risqué	Bien	Mieux	Détection
Variables d'env OS	Secrets CI/CD	Vault	SAST / Scan
export DB_PASS=xxx Lisible par tout le monde sur la machine	GitHub Secrets, GitLab Variables Chiffrés, injectés au runtime	HashiCorp Vault, AWS Secrets Manager Rotation automatique, audit log	truffleHog, gitleaks Détection de secrets dans le code



MOINDRE PRIVILÈGE

Chaque secret n'a accès qu'à ce dont il a besoin

Token GitHub Actions → push registry uniquement

Credentials AWS → deploy sur un seul bucket S3

User DB → read-only sauf pour le service qui écrit

Rotation des credentials régulière

Un token compromis = impact limité à son périmètre.



STRATÉGIES DE DÉPLOIEMENT

- **Blue/Green** : deux envs identiques, bascule instantanée, rollback en 1 clic
- **Canary** : déploiement sur 5% des users d'abord, puis 100% si tout va bien
- **Rolling update** : mise à jour instance par instance, pas de downtime
- **Feature flags** : fonctionnalité activable sans redéployer

Le choix dépend du risque, de la criticité et de l'infrastructure.

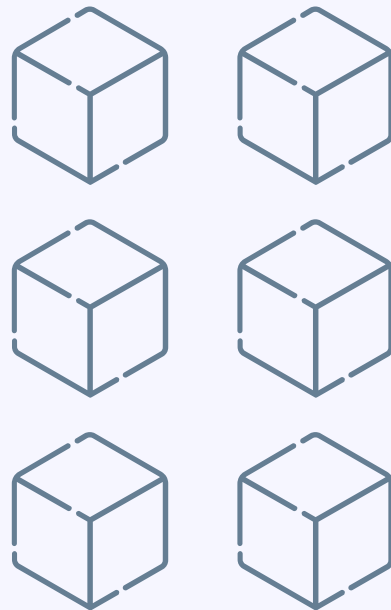
APRÈS LE DÉPLOIEMENT

- **Health checks** : l'app répond-elle correctement après déploiement ?
- **Smoke tests** : quelques requêtes critiques pour valider que rien n'est cassé
- **Alerting** : Prometheus + Grafana
- **Rollback automatique** : si un healthcheck échoue → retour à la version précédente

CAS CONCRET

Problème :

- 5 devs, branches divergentes depuis 2 semaines
- Déploiement SSH manuel un vendredi soir
- Oubli d'une migration SQL → prod cassée
- Rollback manuel de 3h, clients impactés

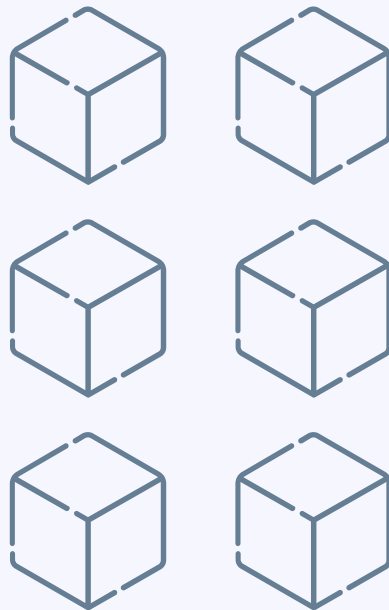


CAS CONCRET

Solution :

- Pipeline GitHub Actions
- Tests + lint à chaque PR (4 min)
- Deploy staging automatique sur merge
- Migration SQL dans le pipeline, rollback si KO
- Deploy prod = 1 clic, vendredi sans stress

Un pipeline = de la sérénité.



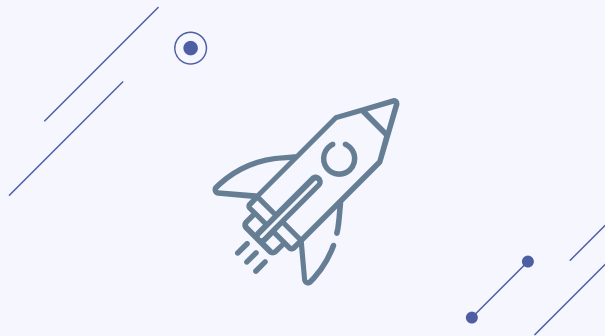
The background is a light blue gradient with various abstract geometric elements in a darker blue. These include thin lines, dots, and small circles scattered across the page, some of which are part of larger, faint geometric shapes like squares and rectangles.

04

TP

Jusqu'où irez-vous ?

Niveau 1 : Novice

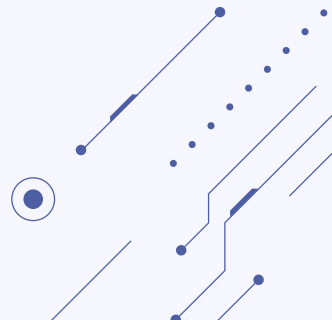


Pipeline

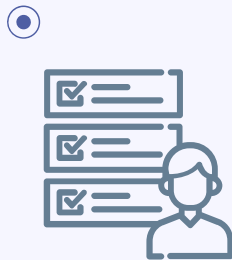
- Créer un dépôt GitHub
- Créer un projet nginx simple
 - .conf nginx
 - index.html
 - Dockerfile
- Écrire un workflow build manuel
 - Vérifier la présence des fichiers
 - Tester la configuration Nginx
- Casser volontairement le nginx pour voir si la pipeline échoue

Rendu

- Repo GitHub/GitLab public
- README : schéma + explication pipeline
- Capture d'écran pipeline fonctionnelle
- Présentation orale vendredi



Niveau 2 : Engineer

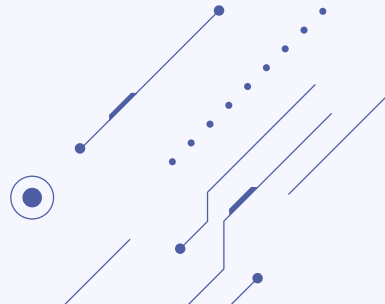


Améliorer la pipeline pour qu'elle :

- se déclenche sur push et pull_request vers la branche main
- vérifie la présence des fichiers essentiels
- teste que le service répond en HTTP
- teste un endpoint /health

Qualité & Sécurité

- Aucun secret dans le code ou les logs



Niveau 3 : Architect



Comme en entreprise

- Publier l'image dans GitHub Container Registry
- Ajouter un tag d'image basé sur le commit SHA
- Ajouter une validation manuelle avant déploiement

